

Laporan Modul 8: Authentication & Authorization

Mata Kuliah: Workshop Web Lanjut

Nama: Ammar

NIM: 2024573010129

Kelas: TI2B

Abstrak

Modul ini membahas konsep autentikasi dan otorisasi dalam Laravel 12 yang menjadi fondasi keamanan aplikasi web modern.

- Autentikasi digunakan untuk memverifikasi identitas pengguna,
- sedangkan otorisasi menentukan hak akses pengguna terhadap sumber daya tertentu.

Praktikum ini menggunakan Laravel Breeze untuk membangun sistem autentikasi dasar dan menerapkan Role-Based Access Control (RBAC) menggunakan middleware, kolom role, serta seeding pengguna dengan peran berbeda. Hasil implementasi menunjukkan bahwa Laravel menyediakan mekanisme yang kuat, fleksibel, dan mudah digunakan untuk mengamankan aplikasi melalui login, proteksi rute, hingga pembatasan akses berbasis peran.

1. Dasar Teori

1.1 Autentikasi (Authentication) dalam Laravel

Autentikasi adalah proses untuk memastikan bahwa pengguna benar-benar memiliki identitas yang sah sebelum dapat mengakses fitur tertentu dalam aplikasi. Laravel menyediakan sistem autentikasi bawaan yang lengkap, aman, dan mudah dikonfigurasi melalui starter kit seperti Laravel Breeze, Jetstream, atau Fortify.

Pada Laravel 12, sistem autentikasi berjalan di atas beberapa komponen inti:

1. Guards

Guard menentukan bagaimana pengguna diautentikasi — misalnya menggunakan sesi (session-based) untuk aplikasi web atau token untuk API. Guard default Laravel adalah web, yang menggunakan session dan cookie terenkripsi.

2. User Providers

Provider menjelaskan bagaimana data pengguna diambil, biasanya melalui model `App\Models\User` yang menggunakan Eloquent ORM.

3. Session & Cookie Authentication Flow

- Pengguna mengirimkan form login
- Laravel memverifikasi kredensial
- Jika valid, Laravel menyimpan session ID dalam cookie yang aman (encrypted & signed)

- Middleware `auth` memeriksa apakah session aktif sebelum memberi akses ke rute yang dilindungi

4. Password Hashing

Laravel menggunakan `bcrypt` atau `argon2id` untuk mengenkripsi password sehingga tidak disimpan sebagai teks asli dalam database.

5. CSRF Protection

Laravel otomatis menjaga keamanan form melalui token CSRF untuk mencegah serangan Cross-Site Request Forgery.

6. Email Verification & Password Reset

Laravel memiliki sistem verifikasi email bawaan, serta mekanisme pemulihan kata sandi melalui token dan link yang dikirimkan melalui email.

Dengan kombinasi fitur-fitur ini, Laravel menyediakan fondasi autentikasi yang aman tanpa konfigurasi kompleks.

1.2 Otorisasi (Authorization) dalam Laravel

Otorisasi menjawab pertanyaan: "Pengguna yang sudah login boleh melakukan apa saja?"

Laravel menyediakan sistem otorisasi yang fleksibel melalui:

1. Gates

Gate adalah fungsi sederhana (closure) yang menentukan izin tertentu. Biasanya digunakan untuk pengecekan akses yang tidak terkait dengan model tertentu.

Contoh: menentukan apakah pengguna boleh mengakses fitur admin.

2. Policies

Policy adalah kelas yang mengelompokkan aturan otorisasi untuk satu model, misalnya `PostPolicy` untuk model `Post`.

Fungsinya antara lain:

- Mengatur siapa yang bisa membuat, mengupdate, atau menghapus data
- Mengelompokkan aturan akses berdasarkan role
- Mengatur akses CRUD secara modular

3. Middleware Authorization

Middleware seperti `auth`, `verified`, atau middleware kustom dapat membatasi akses ke rute tertentu.

Contoh:

```
Route::get('/dashboard')->middleware('auth');
```

4. Role-Based Access Control (RBAC)

RBAC membatasi akses berdasarkan peran (role) seperti:

- Admin
- Manager
- User

Laravel dapat menerapkan RBAC menggunakan:

- Middleware kustom seperti `RoleMiddleware`
- Field `role` di tabel users
- Paket pihak ketiga seperti Spatie Permission

Dengan sistem ini, Laravel mampu menangani otorisasi sederhana hingga kompleks dengan tetap mempertahankan kode yang terstruktur dan mudah dikelola.

1.3 Laravel Breeze

Laravel Breeze adalah paket scaffolding autentikasi sederhana yang menyediakan:

- Rute login, register, logout
- Email verification (opsional)
- Reset password
- Blade siap pakai
- Tailwind CSS untuk tampilan UI modern
- Struktur folder modular untuk autentikasi

Breeze cocok untuk pemula dan proyek yang membutuhkan autentikasi standar tanpa fitur tambahan yang berat seperti Jetstream.

1.4 Middleware dalam Sistem Keamanan

Middleware adalah lapisan pemeriksaan yang berjalan sebelum permintaan (request) mencapai controller. Pada autentikasi dan otorisasi, beberapa middleware penting adalah:

- `auth` → memastikan pengguna login
- `verified` → memastikan email pengguna sudah diverifikasi
- `role` → middleware kustom untuk memeriksa peran pengguna
- `throttle` → mencegah brute force login

Middleware menjadi pondasi penting untuk menjaga aplikasi tetap aman dari akses tidak sah.

1.5 Role-Based Access Control (RBAC)

RBAC adalah pendekatan umum dalam aplikasi modern untuk membatasi akses berdasarkan peran.

Manfaat RBAC:

- Struktur hak akses terorganisasi dan mudah diperluas
- Mempermudah pembagian tugas di aplikasi berskala besar
- Meminimalisir celah keamanan akibat perizinan tidak tepat
- Mencegah pengguna biasa mengakses fitur kritis

Contoh sederhana:

- Admin → akses penuh
- Manager → akses manajemen data
- User → hanya akses fitur umum

Laravel mendukung RBAC dengan sangat fleksibel melalui kombinasi:

- migrasi field role
- middleware role
- seeding pengguna
- view berbeda untuk tiap role

1.6 Keamanan Aplikasi Web dalam Konteks Laravel

Laravel menyediakan fitur keamanan bawaan yang mengurangi risiko umum, antara lain:

- Hashing password
- CSRF protection
- Cross-site scripting (XSS) sanitization
- SQL injection protection melalui query binding
- Session encryption
- Middleware keamanan

Dengan memanfaatkan fitur ini, pengembang dapat membangun aplikasi web yang lebih aman dengan usaha minimal.

2. Langkah-Langkah Praktikum

Praktikum 1 – Autentikasi dan Otorisasi dengan Laravel Breeze

Langkah-langkah:

1. Buat Proyek laravel pada terminal vscode

```
laravel new auth-lab
```

lalu masuk ke dalam folder projek tsb.

```
cd auth-lab
```

pada penginstalan

- pilih mysql sebagai database lalu
- pilih **no** ketika ditanya migration database

2. Buat database TodoDB di Php My Admin atau terminal mysql

```
CREATE DATABASE authlab_db;
```

3. Sesuaikan seperti berikut pada file .env

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=authlab_db
DB_USERNAME=<username database anda>
DB_PASSWORD=<password database anda>
```

lalu bersihkan config cache menggunakan perintah **artisan**:

```
php artisan config:clear
```

4. Instalasi Laravel Breeze

Jalankan perintah berikut pada terminal:

```
composer require laravel/breeze --dev
```

Instal Breeze:

```
php artisan breeze:install
```

Saat proses instalasi:

- Pilih **Blade** sebagai frontend
- Pilih **Yes** untuk dark mode (opsional)

Kemudian jalankan:

```
npm install
php artisan migrate
```

5. Jalankan aplikasi dan tunjukkan hasil di browser.

Jalankan Mysql lalu Untuk menjalankan aplikasi kita bisa menggunakan perintah artisan berikut:

```
php artisan serve
```

lalu ctrl+klik <http://127.0.0.1:8000> sehingga akan diredirect ke web browser.

Di halaman web:

- Klik **Register** untuk membuat akun baru
- Setelah registrasi, pengguna diarahkan ke dashboard
- Lakukan logout lalu login kembali untuk memastikan autentikasi berjalan baik

6. Ganti isi route pada routes/web.php.

```
<?php

use App\Http\Controllers\ProfileController;
use Illuminate\Support\Facades\Route;
use Illuminate\Support\Facades\Auth;

Route::get('/', function () {
    return view('welcome');
});

Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');

Route::middleware('auth')->group(function () {
    Route::get('/profile', [ProfileController::class, 'edit'])->
>name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])->
>name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])->
>name('profile.destroy');

    // Tambahkan rute myprofile baru
    Route::get('/myprofile', function () {
```

```
        return Auth::user();  
    })->name('myprofile');  
});  
  
require __DIR__.'/auth.php';
```

Rute **/myprofile** dilindungi menggunakan middleware **auth**, sehingga hanya user yang sudah login yang dapat mengaksesnya.

7. Pada url tambahkan **/myprofile**

sehingga secara keseluruhan tampak seperti ini:

```
http://localhost:8000/myprofile
```

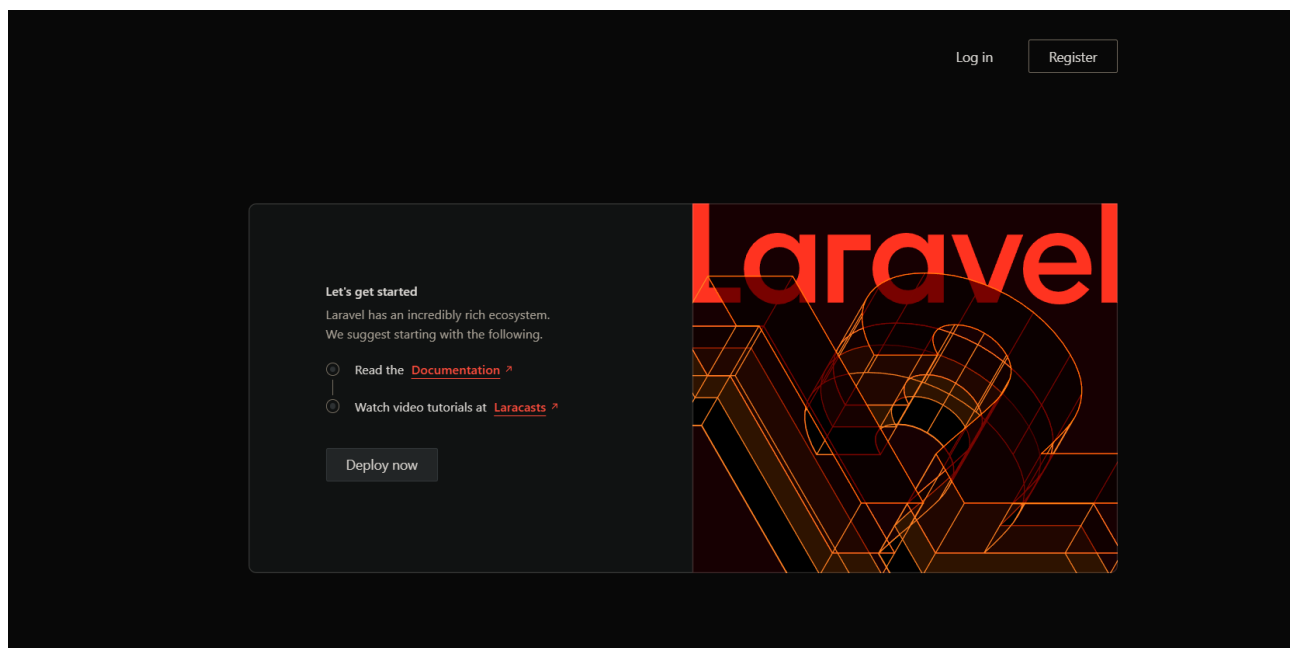
Jika berhasil, aplikasi akan menampilkan data JSON seperti:

- id
- name
- email
- email_verified_at
- created_at
- updated_at

Pengguna juga dapat mengakses halaman edit profil melalui link **Profile**.

Screenshot Hasil:

- Halaman awal



- Register Page



Name

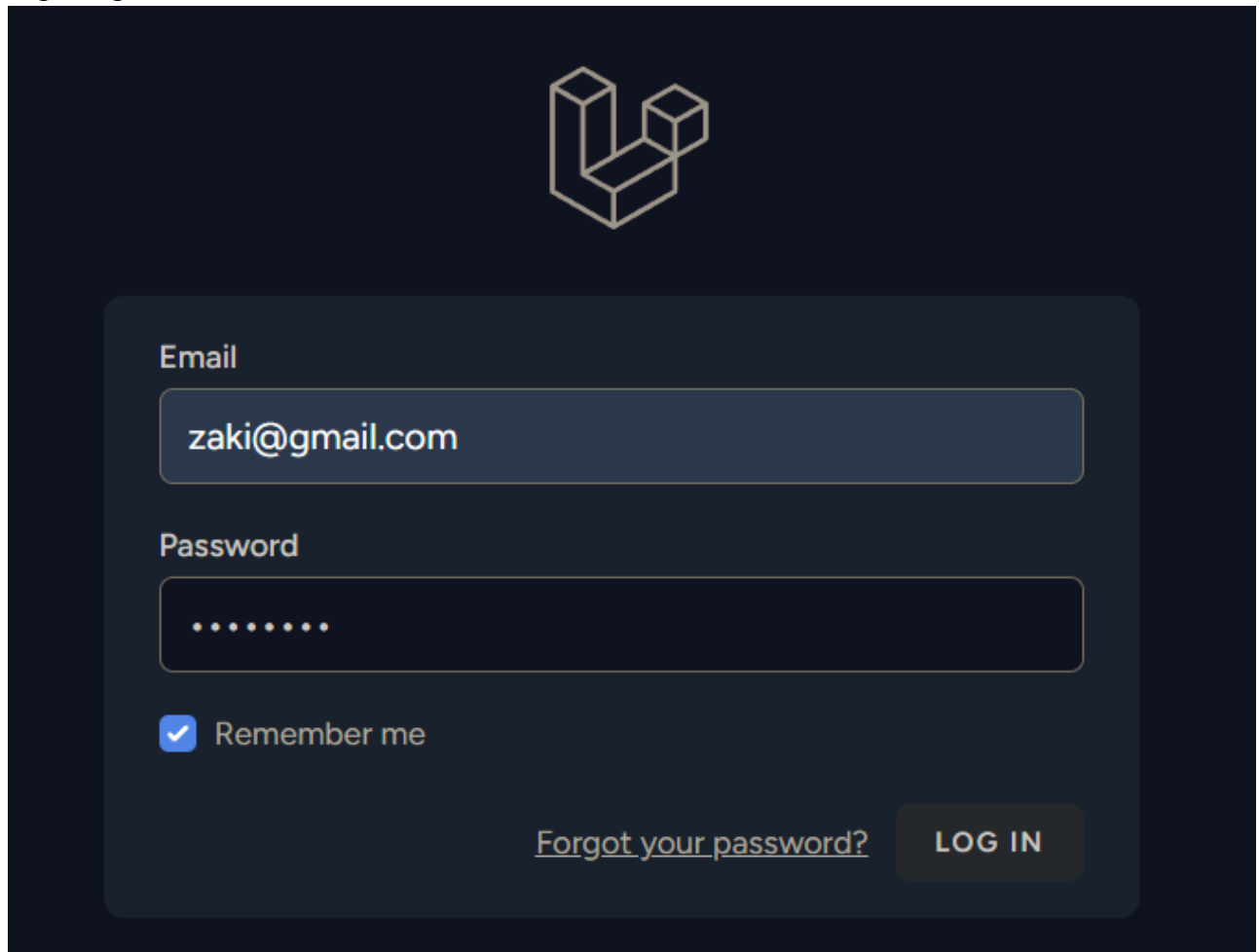
Email

Password

Confirm Password

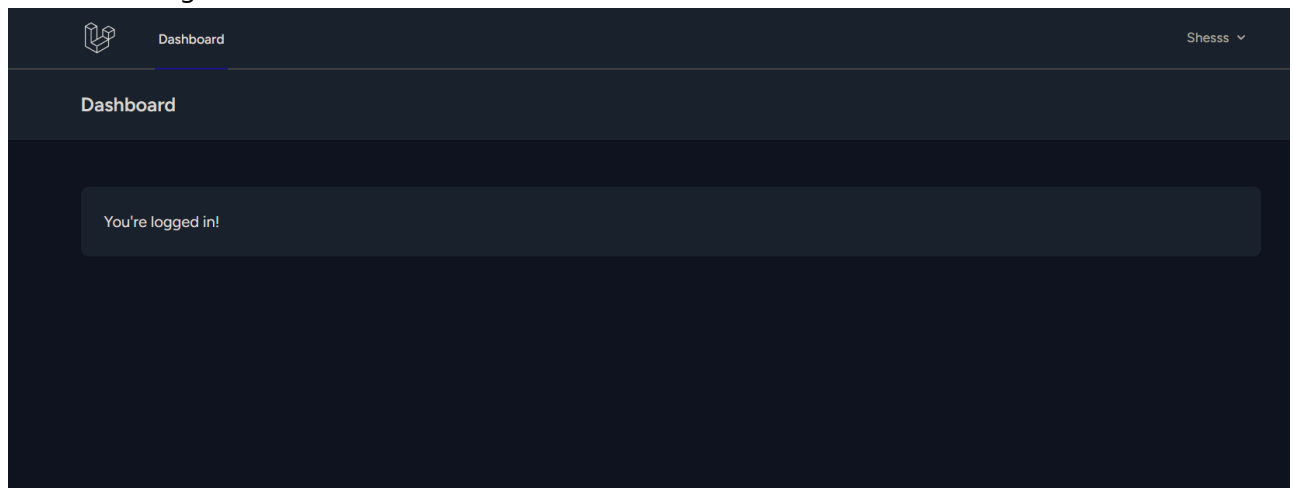
[Already registered?](#) **REGISTER**

- Login Page



A screenshot of a login page with a dark blue background. At the top center is a logo consisting of three isometric cubes. Below the logo is a light blue rounded rectangle containing the login form. The form has two input fields: 'Email' with the text 'zaki@gmail.com' and 'Password' with seven dots. Below the password field is a checkbox labeled 'Remember me' which is checked. At the bottom right of the form is a 'LOG IN' button. To the left of the button is a link that says 'Forgot your password?'.

- Dashboard Page



- Profile

The screenshot shows a dark-themed user profile management interface. At the top, there's a header with a logo, the word 'Dashboard', and a user name 'Shesss' with a dropdown arrow. Below the header, the main section is titled 'Profile'. It contains three distinct panels:

- Profile Information:** Includes a sub-header 'Profile Information' and a description 'Update your account's profile information and email address.' It features input fields for 'Name' (containing 'Shesss') and 'Email' (containing 'zaki@gmail.com'), followed by a 'SAVE' button.
- Update Password:** Includes a sub-header 'Update Password' and a description 'Ensure your account is using a long, random password to stay secure.' It features input fields for 'Current Password', 'New Password', and 'Confirm Password', followed by a 'SAVE' button.
- Delete Account:** Includes a sub-header 'Delete Account' and a description 'Once your account is deleted, all of its resources and data will be permanently deleted. Before deleting your account, please download any data or information that you wish to retain.' It features a prominent red 'DELETE ACCOUNT' button.

- MyProfile (JSON Output)

```
{"id":1,"name":"Shesss","email":"zaki@gmail.com","email_verified_at":null,"created_at":"2025-11-25T05:41:29.000000Z","updated_at":"2025-11-25T05:41:29.000000Z"}
```

2.2 Praktikum 2 – Pembatasan Akses Berdasarkan Peran (Role-Based Access Control)

Langkah-langkah:

1. Buat Proyek laravel pada terminal vscode

```
laravel new role-lab
```

lalu masuk ke dalam folder proyek tsb.

```
cd role-lab
```

pada penginstalan

- pilih mysql sebagai database lalu
- pilih **no** ketika ditanya migration database

2. Buat database TodoDB di Php My Admin atau terminal mysql

```
CREATE DATABASE authrole_db;
```

3. Sesuaikan seperti berikut pada file .env

```
DB_CONNECTION=mysql  
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_DATABASE=authrole_db  
DB_USERNAME=<username database anda>  
DB_PASSWORD=<password database anda>
```

lalu bersihkan config cache menggunakan perintah **artisan**:

```
php artisan config:clear
```

4. Instalasi Laravel Breeze

Jalankan perintah berikut pada terminal:

```
composer require laravel/breeze --dev
```

Instal Breeze:

```
php artisan breeze:install
```

Saat proses instalasi:

- Pilih **Blade** sebagai frontend
- Pilih **Yes** untuk dark mode (opsional)

Kemudian jalankan:

```
npm install
php artisan migrate
```

5. Buat Migrasi

dengan perintah artisan:

```
php artisan make:migration add_role_to_users_table --table=users
```

Buka file yang telah dibuat di `database/migrations/YYYY_MM_DD_add_role_to_users_table.php` lalu ganti dengan:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->string('role')->default('user');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn('role');
        });
    }
};
```

Jalankan Migrasi dengan perintah artisan

```
php artisan migrate
```

6. Buat Seeder untuk pengguna dengan role yang berbeda

Buka file `database/seeder/ProductSeeder.php` lalu ganti dengan:

```
<?php

namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;

class DatabaseSeeder extends Seeder
{
    use WithoutModelEvents;

    /**
     * Seed the application's database.
     */
    public function run(): void
    {
        // User::factory(10)->create();

        // User::factory()->create([
        //     'name' => 'Test User',
        //     'email' => 'test@example.com',
        // ]);

        User::create([
            'name' => 'Admin User',
            'email' => 'admin@ilmudata.id',
            'password' => Hash::make('password123'),
            'role' => 'admin',
        ]);

        User::create([
            'name' => 'Manager User',
            'email' => 'manager@ilmudata.id',
            'password' => Hash::make('password123'),
            'role' => 'manager',
        ]);

        User::create([
            'name' => 'General User',
            'email' => 'user@ilmudata.id',
            'password' => Hash::make('password123'),
            'role' => 'user',
        ]);
    }
}
```

Jalankan Seeder dengan perintah artisan

```
php artisan db:seed
```

7. Membuat role middleware

dengan perintah artisan:

```
php artisan make:middleware RoleMiddleware
```

Buka file yang telah dibuat di `app\Http\Middleware\RoleMiddleware.php` lalu ganti dengan:

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class RoleMiddleware
{
    /**
     * Handle an incoming request.
     *
     * @param  \Closure(\Illuminate\Http\Request):
     * (\Symfony\Component\HttpFoundation\Response)  $next
     */
    public function handle(Request $request, Closure $next, string $role):
    Response
    {
        if ($request->user() && $request->user()->role === $role) {
            return $next($request);
        }

        abort(403, 'Unauthorized');
    }
}
```

8. Ganti isi route pada routes/web.php.

```
<?php

use App\Http\Controllers\ProfileController;
```

```

use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return redirect()->route('login');
});

Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');

Route::middleware('auth')->group(function () {
    Route::get('/profile', [ProfileController::class, 'edit'])->
    >name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])->
    >name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])->
    >name('profile.destroy');

    // Rute untuk semua user yang terautentikasi
    Route::get('/all', function () {
        return view('all');
    });

    // Rute untuk admin
    Route::get('/admin', function () {
        return view('admin');
    })->middleware('role:admin');

    // Rute untuk semua user yang terautentikasi
    Route::get('/manager', function () {
        return view('manager');
    })->middleware('role:manager');

    // Rute untuk semua user yang terautentikasi
    Route::get('/user', function () {
        return view('user');
    })->middleware('role:user');
});

require __DIR__.'/auth.php';

```

9. Buat view di `resources\views`.

dengan perintah touch:

```

touch resources/views/admin.blade.php
touch resources/views/all.blade.php
touch resources/views/user.blade.php
touch resources/views/manager.blade.php

```

Masuk ke file `resources/views/admin.blade.php` dan isikan code berikut:

```
<x-app-layout>
  <x-slot name="header">
    <h2 class="font-semibold text-xl text-gray-800 dark:text-gray-200
leading-tight">
      {{ __('Admin Dashboard') }}
    </h2>
  </x-slot>

  <div class="py-12">
    <div class="max-w-7xl mx-auto sm:px-6 lg:px-8">
      <div class="bg-white dark:bg-gray-800 overflow-hidden shadow-sm
sm:rounded-lg">
        <div class="p-6 text-gray-900 dark:text-gray-100">
          {{ __("Welcome, Admin! You have full access.") }}
        </div>
      </div>
    </div>
  </div>
</x-app-layout>
```

Masuk ke file `resources/views/manager.blade.php` dan isikan code berikut:

```
<x-app-layout>
  <x-slot name="header">
    <h2 class="font-semibold text-xl text-gray-800 dark:text-gray-200
leading-tight">
      {{ __('Manager Dashboard') }}
    </h2>
  </x-slot>

  <div class="py-12">
    <div class="max-w-7xl mx-auto sm:px-6 lg:px-8">
      <div class="bg-white dark:bg-gray-800 overflow-hidden shadow-sm
sm:rounded-lg">
        <div class="p-6 text-gray-900 dark:text-gray-100">
          {{ __("Welcome, Manager! You can manage and monitor
resources.") }}
        </div>
      </div>
    </div>
  </div>
</x-app-layout>
```

Masuk ke file `resources/views/all.blade.php` dan isikan code berikut:

```
<x-app-layout>
  <x-slot name="header">
    <h2 class="font-semibold text-xl text-gray-800 dark:text-gray-200
leading-tight">
      {{ __( 'General Dashboard' ) }}
    </h2>
  </x-slot>

  <div class="py-12">
    <div class="max-w-7xl mx-auto sm:px-6 lg:px-8">
      <div class="bg-white dark:bg-gray-800 overflow-hidden shadow-sm
sm:rounded-lg">
        <div class="p-6 text-gray-900 dark:text-gray-100">
          {{ __( "Welcome! This view is accessible by all
authenticated roles." ) }}
        </div>
      </div>
    </div>
  </div>
</x-app-layout>
```

Masuk ke file `resources/views/user.blade.php` dan isikan code berikut:

```
<x-app-layout>
  <x-slot name="header">
    <h2 class="font-semibold text-xl text-gray-800 dark:text-gray-200
leading-tight">
      {{ __( 'User Dashboard' ) }}
    </h2>
  </x-slot>

  <div class="py-12">
    <div class="max-w-7xl mx-auto sm:px-6 lg:px-8">
      <div class="bg-white dark:bg-gray-800 overflow-hidden shadow-sm
sm:rounded-lg">
        <div class="p-6 text-gray-900 dark:text-gray-100">
          {{ __( "Welcome, User! You have limited access." ) }}
        </div>
      </div>
    </div>
  </div>
</x-app-layout>
```

10. Jalankan aplikasi dan tunjukkan hasil di browser.

Jalankan Mysql lalu Untuk menjalankan aplikasi kita bisa menggunakan perintah artisan berikut:

```
php artisan serve
```

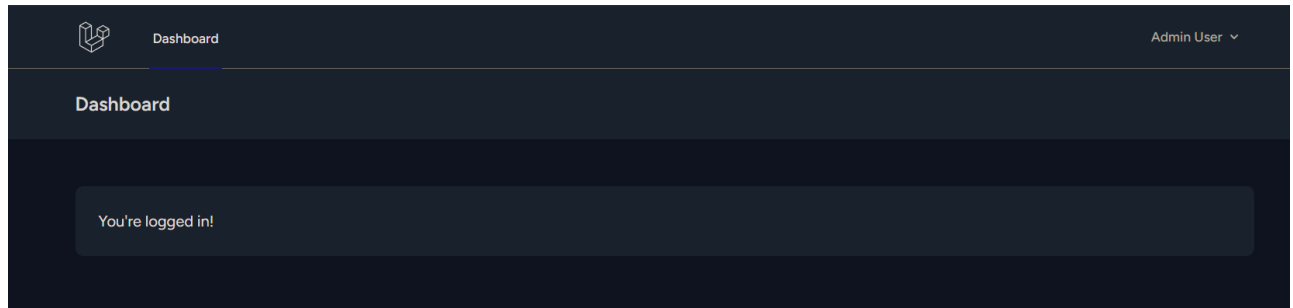
lalu ctrl+klik <http://127.0.0.1:8000> sehingga akan diredirect ke web browser.

login menggunakan pengguna yang telah disediakan:

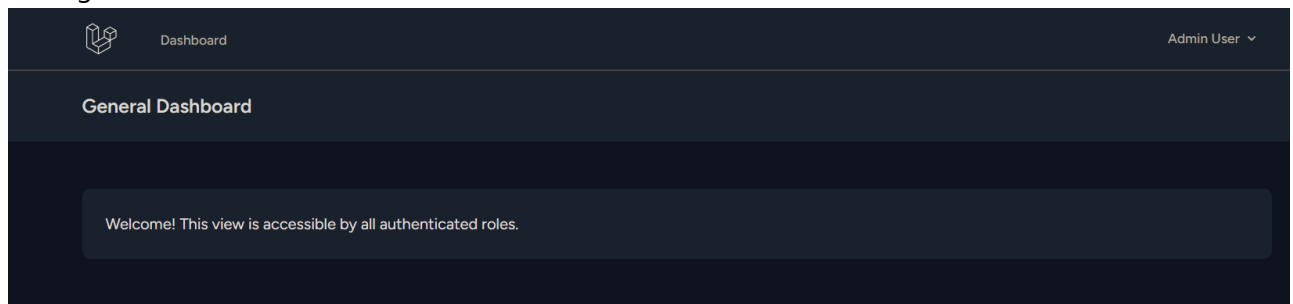
- Admin: admin@ilmudata.id / password123
- Manager: manager@ilmudata.id / password123
- User: user@ilmudata.id / password123

Screenshot Hasil:

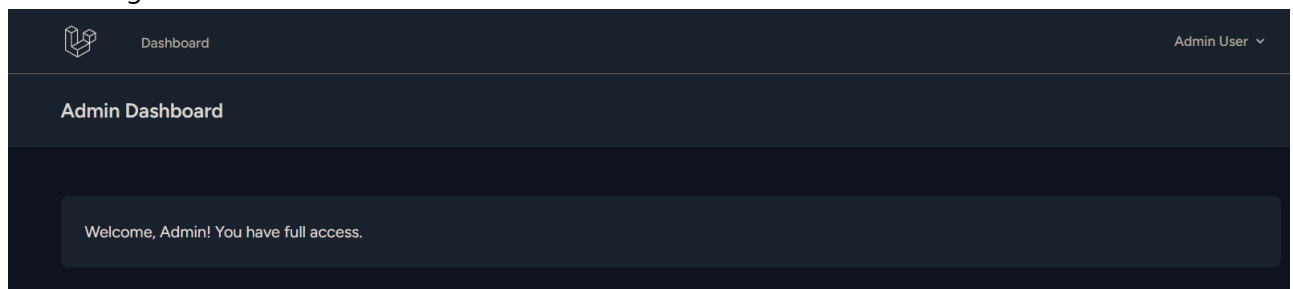
- Dashboard Page



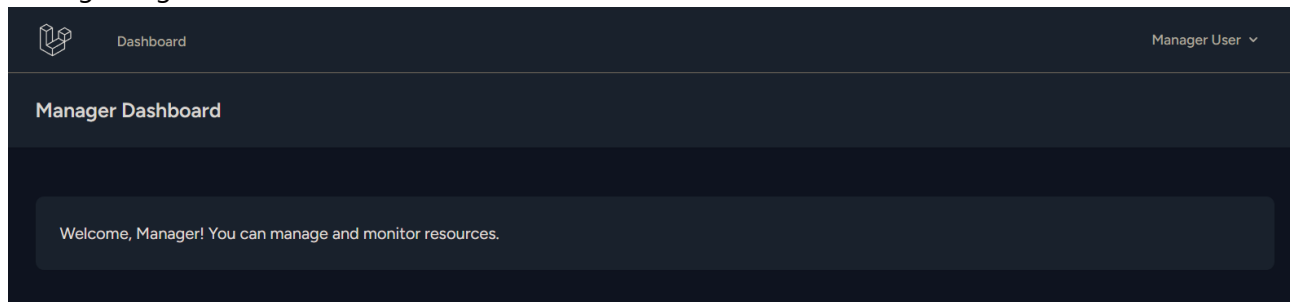
- All Page



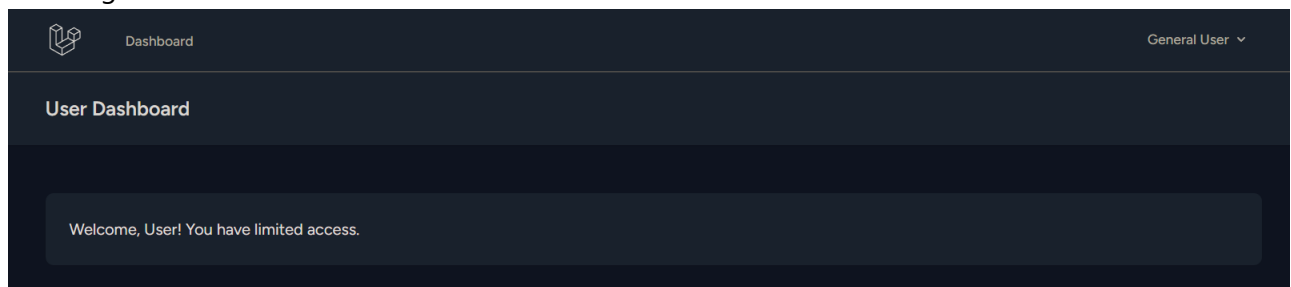
- Admin Page



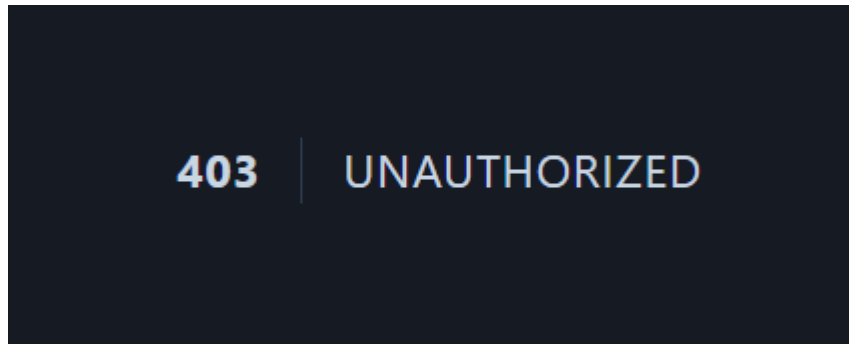
- Manager Page



- User Page



- Unauthorized (jika tidak sesuai role)



3. Hasil dan Pembahasan

Implementasi autentikasi dasar dengan Breeze berhasil, ditunjukkan dengan berfungsinya proses:

- registrasi,
- login,
- logout,
- serta proteksi rute menggunakan middleware.

Rute `/myprofile` menampilkan informasi pengguna yang sedang login, membuktikan sistem autentikasi berjalan dengan baik.

Pada praktikum Role, sistem berhasil membedakan hak akses setiap pengguna berdasarkan perannya.

- Admin hanya dapat membuka halaman admin,
- manager hanya dapat membuka halaman manager,
- dan begitu pula user.
- Sementara itu, halaman umum(all) dapat diakses semua pengguna yang telah login.

Hal ini menunjukkan bahwa middleware role bekerja sesuai tujuan, dan mekanisme otorisasi Laravel sangat efektif dalam membatasi akses.

4. Kesimpulan

Laporan ini menunjukkan bahwa Laravel menyediakan sistem autentikasi dan otorisasi yang kuat, mudah digunakan, serta dapat diperluas sesuai kebutuhan aplikasi. Breeze mempermudah pembuatan fitur autentikasi standar, sementara middleware dan role memungkinkan penerapan kontrol akses yang lebih granular. Praktikum membuktikan bahwa seluruh mekanisme bekerja dengan efektif dalam mengamankan aplikasi dan membatasi akses sesuai peran.

5. Referensi

- Laravel Documentation – Authentication: <https://laravel.com/docs/12.x/authentication>
- Laravel Documentation – Authorization: <https://laravel.com/docs/12.x/authorization>
- Laravel Breeze – <https://laravel.com/docs/12.x/starter-kits#laravel-breeze>
- Modul 8 – Authentication & Authorization – hackmd.io/@mohdrzu/BypBawklWg